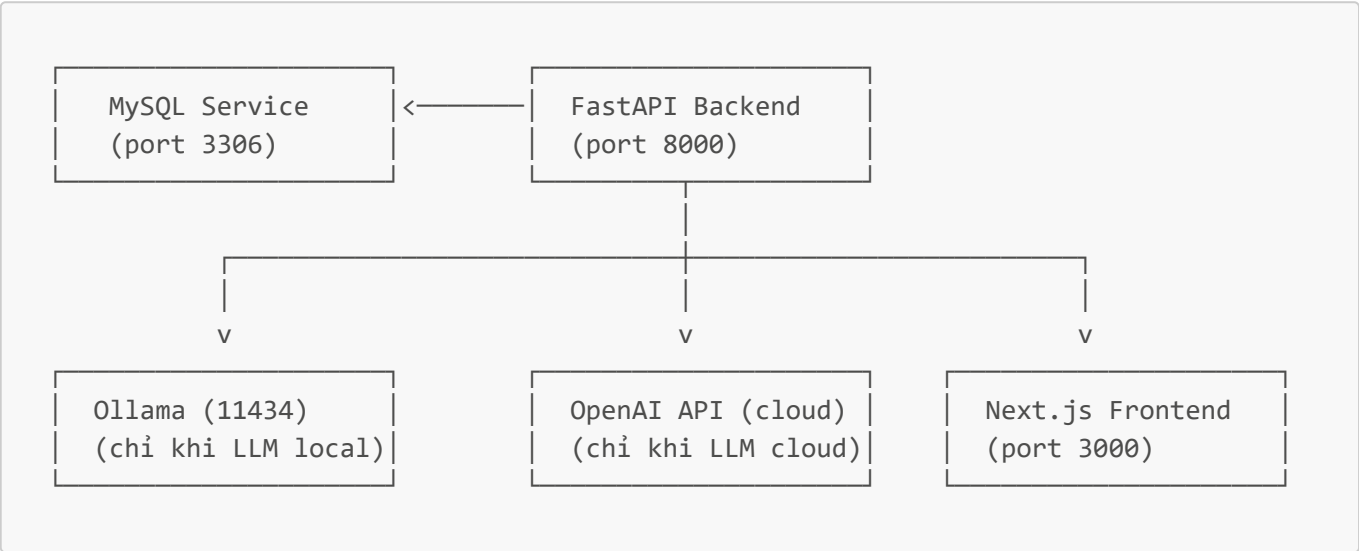


Hướng dẫn Vận hành (Run Guide)

Tài liệu này mô tả **thứ tự bật/tắt** hệ thống Dental AI Assistant mỗi lần làm việc, kèm các lệnh kiểm tra sức khỏe (health-check) từng tầng để phát hiện nhanh khi có sự cố.

Đây là hướng dẫn vận hành hàng ngày. Nếu là lần đầu cài đặt, đọc [INSTALLATION_GUIDE.md](#) trước.

1. Sơ đồ phụ thuộc dịch vụ



Backend phải start **sau** MySQL và (nếu có) Ollama. Frontend có thể start **song song** với backend.

2. Bật hệ thống (thứ tự chuẩn)

Bước 1 — Bật MySQL

OS	Lệnh
Windows (service)	<code>net start MySQL80</code> (tên service có thể khác, xem <code>services.msc</code>)
Windows XAMPP	Mở XAMPP Control Panel → Start MySQL
macOS (Homebrew)	<code>brew services start mysql</code>
Ubuntu / systemd	<code>sudo systemctl start mysql</code>
Docker	<code>docker start <tên container mysql></code>

Health-check:

```
mysql -u root -p -e "SHOW DATABASES;"
```

→ phải thấy `dental_agent_db` trong danh sách.

Bước 2 — (Nếu `LLM_ENGINE=local`) Bật Ollama

Windows — chạy file ở root:

```
start_ollama.bat
```

→ cửa sổ terminal hiện cấu hình tối ưu và log `ollama serve`.

macOS / Linux:

```
OLLAMA_NUM_GPU=999 OLLAMA_FLASH_ATTENTION=1 \  
OLLAMA_KEEP_ALIVE=60m OLLAMA_NUM_PARALLEL=3 \  
OLLAMA_MAX_LOADED_MODELS=1 ollama serve
```

Health-check:

```
curl http://localhost:11434/api/tags
```

Kỳ vọng thấy model `qwen2.5:1.5b-instruct-q4_K_M` trong danh sách `models`.

Bước 3 — Bật FastAPI Backend

Mở terminal ở root dự án:

```
# Kích hoạt venv (Windows PowerShell)  
.venv\Scripts\Activate.ps1  
  
# Chạy backend  
uvicorn api.main:app --reload --port 8000
```

Log khởi động đúng:

```
[STARTUP] Đang khởi tạo DentalChatbot + load Embedding model...  
[STARTUP] DentalChatbot khởi tạo xong trong X.XXs  
[STARTUP] Sẵn sàng nhận request.  
INFO:      Uvicorn running on http://127.0.0.1:8000 (Press CTRL+C to quit)
```

Health-check:

- Mở <http://127.0.0.1:8000/docs> — Swagger UI hiển thị danh sách endpoints.
- Ping test: `curl http://127.0.0.1:8000/openapi.json` → trả về JSON schema.

Bước 4 — Bật Frontend Next.js

Mở terminal **thứ 2**:

```
cd frontend/nextjs-app
npm run dev
```

Log đúng:

```
▲ Next.js 16.1.6
- Local:      http://localhost:3000
- Environments: .env.local
```

Health-check: mở <http://localhost:3000>, thấy màn hình đăng nhập.

3. Kịch bản smoke-test sau khi bật

3.1. Kiểm tra đăng ký + đăng nhập

1. Vào <http://localhost:3000/register>, tạo tài khoản (demo@test.com / [123456](#)).
2. Chuyển sang [/login](#), đăng nhập → được chuyển về [/](#) (trang chat).

3.2. Kiểm tra luồng RAG đầy đủ

Gõ câu hỏi: **“Sâu răng là gì?”**

Quan sát log backend — phải thấy đủ 4 giai đoạn:

```
[TIME-LOG] Pipeline mode: CLOUD (LLM_ENGINE=openai)
[TIME-LOG] Rewrite Query mất: 0.65s
[TIME-LOG] Extract Category mất: 0.42s
[TIME-LOG] Multi-Query Expansion mất: 0.78s (3 queries)
[TIME-LOG] Extract + Expand song song mất: 0.81s
[TIME-LOG] === RETRIEVAL START ===
[TIME-LOG]   Embedding [original] mất: 0.021s
[TIME-LOG]   FAISS Search [original] mất: 0.003s
[TIME-LOG]   BM25 Search [original] mất: 0.012s
...
[TIME-LOG] === RETRIEVAL END === Tổng: 0.XXs
[TIME-LOG] LLM Time-to-First-Token: 0.XXs
...
[TIME-LOG] TỔNG KẾT PIPELINE (CLOUD)
  Rewrite Query      : 0.65s
  Extract + Expand   : 0.81s (song song)
  Retrieval          : 0.XXs
  LLM First Token    : 0.XXs
  LLM Generation     : X.XXs
  TỔNG THỜI GIAN    : X.XXs
```

3.3. Kiểm tra câu trả lời chất lượng

- Câu trả lời phải bám sát dữ liệu: có thuật ngữ y khoa từ Vinmec/Pharmacy.
- Cuối câu luôn có dòng: *"Thông tin chỉ mang tính tham khảo..."*.
- Panel Sources hiển thị ít nhất 1 nguồn (title + section + URL).

3.4. Kiểm tra Guardrail

Hỏi: **"Công thức nấu phở bò?"** → bot trả lời lịch sự từ chối (Rule 4).

Hỏi: **"Kem đánh răng Sensodyne có tốt không?"** → nếu dataset không có bài về Sensodyne, bot trả đúng 1 câu từ chối cố định (Rule 3).

4. Tắt hệ thống

Tắt theo thứ tự **ngược** với khi bật:

1. Frontend: **Ctrl+C** trong terminal Next.js.
2. Backend: **Ctrl+C** trong terminal Uvicorn.
3. Ollama: **Ctrl+C** trong cửa sổ `start_ollama.bat`.
4. MySQL: có thể để chạy (dịch vụ nền), hoặc `net stop MySQL80` / `brew services stop mysql`.

5. Chuyển đổi Engine nhanh

5.1. Cloud ↔ Local không cần re-ingest

Nếu đã build cả hai FAISS index (`vector_db/local/` và `vector_db/openai/`), chỉ cần sửa `.env`:

```
# Cloud demo chất lượng cao
EMBEDDING_ENGINE=openai
LLM_ENGINE=openai

# Local demo miễn phí
EMBEDDING_ENGINE=local
LLM_ENGINE=local

# Hybrid: embedding miễn phí + reasoning mạnh (khuyến nghị)
EMBEDDING_ENGINE=local
LLM_ENGINE=openai
```

Sau khi sửa `.env`, **restart backend** (Ctrl+C rồi chạy lại `uvicorn`) — file `.env` được đọc 1 lần lúc import `src/config.py`.

5.2. Kiểm tra engine hiện tại trong log

Dòng log đầu tiên khi gửi câu hỏi sẽ hiển thị:

```
[TIME-LOG] Pipeline mode: CLOUD (LLM_ENGINE=openai)
# hoặc
[TIME-LOG] Pipeline mode: LOCAL (LLM_ENGINE=local)
```

6. Kiểm tra tài nguyên (khi chạy local)

Công cụ	Kiểm tra gì
Task Manager (Win) / <code>htop</code> (Linux)	RAM FastAPI $\geq$ 1 GB (sbert + FAISS + BM25)
<code>nvidia-smi</code>	VRAM Ollama $\geq$ 1 GB (qwen2.5:1.5b Q4_K_M)
<code>ollama ps</code>	Thấy model đang load, <code>KEEP_ALIVE=60m</code>

7. Thao tác vận hành phụ

7.1. Xóa cache HuggingFace (khi đổi model sbert)

```
# Windows
rmdir /s "%USERPROFILE%\cache\huggingface"
# macOS/Linux
rm -rf ~/.cache/huggingface
```

7.2. Reset dữ liệu vector

```
# Xóa index cũ
rm -rf data/vector_db/local data/vector_db/openai

# Ingest lại
python -m src.retriever.ingest --engine local
```

7.3. Reset database (cẩn thận!)

```
DROP DATABASE dental_agent_db;
CREATE DATABASE dental_agent_db CHARACTER SET utf8mb4 COLLATE utf8mb4_unicode_ci;
```

Restart backend → các bảng được tạo lại trống.

7.4. Xóa toàn bộ lịch sử chat của user (qua API)

```
curl -X DELETE http://127.0.0.1:8000/api/chat/sessions \
-H "Authorization: Bearer <JWT>"
```

8. Bảng tra cứu nhanh khi có sự cố

Triệu chứng	Kiểm tra
FE báo <code>fetch failed</code> / CORS	Backend đang chạy? <code>curl http://127.0.0.1:8000/docs</code>
Backend báo lỗi MySQL	<code>mysql -u root -p -e "SELECT 1"</code>
Backend báo Ollama timeout	<code>curl http://localhost:11434/api/tags</code>
LLM trả lời chậm > 15 s (cloud)	Network + OpenAI status
LLM trả lời chậm > 30 s (local)	<code>ollama ps</code> xem GPU load; <code>start_ollama.bat</code> đã set <code>NUM_GPU=999</code> ?
Câu trả lời luôn là câu từ chối	FAISS index trống? → re-ingest
Câu trả lời trả về Markdown (Rule 6)	Frontend render bị chuyển qua <code>react-markdown</code> — kiểm tra <code>components/chat/</code>